

Soluciones a los problemas: Clase 8 - Algoritmos voraces (greedy) 2



Problemas: Clase 9 – Técnicas de Sumas Parciales y Aplicaciones en Intervalos

Máxima longitud

```
# Leer el primer numero desde la entrada
a = int(input())
# Leer el segundo numero desde la entrada
b = int(input())
# Leer el tercer numero desde la entrada
c = int(input())
# Imprimir el mayor de los tres valores usando la funcion max
print(max(a, b, c))
```

¿Número Par o Impar?

```
# Leer el numero entero desde la entrada
a = int(input())
# Comprobar si el numero es par usando el operador modulo
if a % 2 == 0:
    # Si el residuo es 0, el numero es par
    print("Even")
else:
    # Si no es 0, el numero es impar
    print("Odd")
```

Cálculo de Prefix Sums

```
# Leer n (tamano del arreglo)
n = int(input())
# Leer la linea con n enteros y convertirlos a una lista de enteros
li = [int(x) for x in input().split()]
# Crear el arreglo prefix del mismo tamano, inicializado con ceros
prefix = [0] * len(li)
# El primer elemento del prefix es el primer elemento del arreglo original
prefix[0] = li[0]
# Calcular las sumas acumuladas desde el indice 1 hasta el final
for i in range(1, len(li)):
    # prefix[i] es el elemento actual mas la suma acumulada anterior
    prefix[i] = li[i] + prefix[i-1]
# Imprimir todos los valores del arreglo prefix separados por espacio
```

```
for i in range(len(prefix)):
    print(prefix[i], end = " ")
```

Índice de Equilibrio en un Arreglo

```
# Leer n (tamano del arreglo)
n = int(input())
# Leer la lista de enteros
li = [int(x) for x in input().split()]
# Crear arreglo de prefix sums
prefix = [0] * len(li)
prefix[0] = li[0]
# Calcular prefix sums
for i in range(1, len(li)):
    prefix[i] = prefix[i-1] + li[i]
# Buscar el primer indice de equilibrio
for i in range(len(li)):
    # Caso especial: indice 0
    if i == 0 and prefix[-1] - li[0] == 0:
        print(0)
        break
    # Caso especial: ultimo indice
    elif i == n-1 and prefix[-1] - li[-1] == 0:
        print(n-1)
        break
    # Caso general: suma izquierda igual a suma derecha
    if prefix[i-1] == prefix[-1] - prefix[i]:
        print(i)
        break
else:
    # Si no existe indice de equilibrio
    print(-1)
```

Dividir un arreglo en dos subarreglos con suma igual

```
# Leer n (tamano del arreglo)
n = int(input())
# Leer la lista de enteros positivos
li = list(map(int, input().split()))
# Calcular la suma total del arreglo
suma = sum(li)
# Variable para llevar la suma acumulada desde el inicio
acumulado = 0
# Recorrer el arreglo buscando el punto donde ambas sumas sean iguales
for i in range(n):
    # Restar el elemento actual de la suma total (suma del lado derecho)
    suma -= li[i]
    # Agregar el elemento actual a la suma acumulada (lado izquierdo)
    acumulado += li[i]
```

```
# Verificar si las sumas izquierda y derecha son iguales
if suma == acumulado:
    # Imprimir primer subarreglo
    print(" ".join(map(str, li[:i+1])))
    # Imprimir segundo subarreglo
    print(" ".join(map(str, li[i+1:])))
    break
else:
    # Si nunca se encontro una division valida
    print("No es Posible")
```

Calcular la media entera en rangos de un arreglo

```
# Leer n (tamano del arreglo) y q (numero de consultas)
n, q = list(map(int, input().split()))
# Leer el arreglo de n enteros
li = list(map(int, input().split()))
# Crear arreglo de prefix sums
psum = [0] * n
psum[0] = li[0]
# Calcular prefix sums
for i in range(1, n):
    psum[i] = psum[i-1] + li[i]
# Procesar cada consulta
for i in range(q):
    a, b = list(map(int, input().split()))
    # Caso en que el rango empieza en 1
    if a == 1:
        print(psum[b-1] // b)
    else:
        # Suma del rango usando prefix sums
        print((psum[b-1] - psum[a-2]) // (b - a + 1))
```

Recuperar el arreglo original a partir del arreglo de sumas parciales (prefix sums)

```
# Leer n (tamano del arreglo de sumas parciales)
n = int(input())
# Leer el arreglo presum
li = list(map(int, input().split()))
# Variable para llevar la suma acumulada anterior
tot = 0
# Reconstruir el arreglo original
for i in range(n):
    # El valor original es la diferencia entre la suma actual y la acumulada
    # previa
    print(li[i] - tot, end=" ")
    # Actualizar el valor en la lista para mantener la resta correcta
    tot = li[i]
```

```
li[i] = li[i] - tot
# Actualizar la suma acumulada
tot += li[i]
```

Producto del arreglo excepto el elemento actual

```
# Leer n (tamano del arreglo)
n = int(input())
# Leer la lista de enteros
li = [int(x) for x in input().split()]
# Arreglo de productos prefix (producto desde el inicio hasta i)
psum = [1] * n
for i in range(n):
    if i == 0:
        psum[i] = li[i]                # Primer producto
    else:
        psum[i] = li[i] * psum[i-1]    # Producto acumulado
# Arreglo de productos suffix (producto desde el final hasta i)
bsum = [1] * n
for i in range(n-1, -1, -1):
    if i == n-1:
        bsum[i] = li[i]                # Ultimo producto
    else:
        bsum[i] = li[i] * bsum[i+1]    # Producto acumulado desde atras
# Construir el resultado: producto de todos menos arr[i]
for i in range(n):
    if i == 0:
        print(bsum[i+1], end=" ")      # Solo derecha
    elif i == n-1:
        print(psum[i-1], end=" ")      # Solo izquierda
    else:
        print(psum[i-1] * bsum[i+1], end=" ") # Izquierda * derecha
```